

Assignment 5

COSC 2P89 Code Due: Friday, Dec 10, 2021 at 12:00 pm on Sakai (no extensions)

The time has come! You finally get to combine what you've learned over the course to create a reasonably well-featured website. Revisiting A1, the premise is your own personal website. This time, you will extend it using newer concepts in the class, including registering/forms, dynamic design, security, and databases. The informal requirements are as such:

You're making a site with (what appears to be) multiple pages. There should be common elements (notions of theming, banners, etc.) across the site. There will be several primary pages: a **Home** page, displaying basic information about yourself (background, research/academic interests); a **Courses** page, which displays a list of courses you are currently taking (or a longer list of all courses you have taken); an **About me**, which provides a deeper summary of you and your interests (they may even be hobbies!), a **CV** page, which displays the latest version of your CV (this may be just a pretty and high resolution image of a CV you already have, or you can format it as textual display on the webpage itself).

Now, as additional pages (these one naturally may not always be expected from a personal webpage, so feel free to remove these later when completing your own professional pages), you will have a **Past Projects** page, in which you may include links to past projects you have completed, but must, **crucially**, include links to your past four assignments in our class.

Relating to this page, you will have a **sign-up** and **log-in** button (can be alongside the menu that takes you to the mentioned sections of your website), which allows users on your website to sign-up/log-in **in order** to access the content in your **Past Projects** page. In addition, if the user is *not* currently logged in, your **Past Projects** page will always succeed a login form (otherwise they will be directed to it automatically), where the user must enter a valid (i.e., registered) username and password to access the page. If they fail, they are prompted with some error and may retry. I'm asking you to use **asynchronous** communication for this.

Security: We don't want the user to need to log-in *every* time they want to visit your **Past Projects**. As shown in lecture 19, this can be accomplished by passing the username and password to this page, after having logged in once, with the magic of PHP and hidden forms. However, this isn't very secure as this information will be transferred through packets upon every request; I am therefore wanting you to store the username and password locally on the client side using **session storage** (and not web storage) in order for it to neither be transferred nor be permanent.

You should **not** use this template of mine as a copying mechanism, and I'm asking the TAs to pay attention to that, but you can use my personal webpage here as an aesthetic example of what a themed personal webpage *could* look like: <http://cosc.brocku.ca/~aemami/>. Obviously, it doesn't abide by a lot of the requirements for this assignment.

Your webpage should also be responsive – again, responsive design means that your webpage must work on different display sizes. Your web page must work, as a minimum, on cellphone, tablet and desktop display sizes. The TA will test this by reducing the browser window size on their PC. The TA will not actually test on a cellphone or a tablet.

BONUS (see rubric): Now, what if I asked you to do this all in a single .html file (a *single page*)? You may also see this as having 1 .html file *and* 1 .php file, or a single .php file. Also, what if you stored a ticket (randomly generated string) for each username/password pair in session storage?

Requirements and tips

To clarify what's expected of you:

General:

- There is obviously database work here. It's assumed you'll be writing all server-side scripting in Python (and if you so wish, C), but if there are elements you can write in PHP, the choice is yours.
 - It's certainly *easier* to stick with one language
 - Please use CSV as your database file to store username/password info.
- You are not allowed to use WYSIWIG (code generating tools).
- Pages should somehow display the current login status.

Signing up:

- *Somewhere*, you'll need the ability to sign up for a new account
- The user chooses a name and password
 - That name can't conflict with existing names

General:

- Pages should somehow display the current login status
- Your TAs will be checking out your databases as well.

RUBRIC /30 (without bonus)

Site: /6

- 1 for home page
- 1 for courses page
- 1 for about me page
- 1 for CV page
- 1 for past projects page

Functional components: /6

- 1 for reasonable use of Ajax, to avoid reloading pages when it isn't necessary.
- 3 for correct sign-up/log-in logic
 - Will only registered users be able to access past projects page?
 - Do you prevent against duplicate usernames?
 - Does signing up correctly store the required info into the CSV?
- 1 for CSV file provided with username/password.
- 1 for login status displayed on pages

Users: /3

- 1 for displaying 'sign out'/'sign in', according to login status (i.e. 1 mark for the client-side work)
- 1 for managing signed-in/out status (deciding what to do/show depending on status on the server)
- 1 for creating/organizing user accounts/profiles, *including the 'sign up' page*

Aesthetics: /8

- 3 for overall design of the page.
- 2 if the web page seems like a believable, and functional personal webpage (via relevant content, images, etc).
- 3 for responsiveness.

Security: /4

- 2 for correctly storing username + password information in session storage.
- 1 users should not be able to access the Past Projects page by just typing the destination into the url without having logged in.
- 1 for no excessively insecure permissions for files in server side (see lecture 19 about chmod).

Additional considerations: /3

- 1 for correctness/unbreakability of server-side content in general
- 1 for correctness/unbreakability of client-side content in general
- 1 for reasonableness of implementation (this is separate from correctness)

Bonus: /15

(Yes, you may end up with >100% on this, which contributes positively to your overall class grade)

- 10 if you manage to make your webpage a **single-page website**.
- 2 if you manage to store username/password as tickets.
- 3 for user-selectable colour themes
 - It's up to you whether it's a server-side account thing, a localStorage thing, a cookie thing, etc

Submission

You'll be submitting through Sakai.

Bundle everything up into a single `.zip` file, and submit that. This mean all your `.html/.php` files, scripts, and possibly `.c` files as well.

As you have used CSV files as your database, include these in the submission. If you need anything else to help your marker understand what you did, you might want to include that, too.

It's highly advisable to include (brief) instructions for how to access/use your page. I'd suggest something like a `readme.pdf` in the submission.

The marker should be able to access your page at:
<https://www.cosc.brocku.ca/~yourusername/A5/>

Reminder: regardless of what's on the marking scheme, if the markers can't test your page, you can't expect a grade!